

Technical Design Document

FX Trading Tracker Platform | Architecture, APIs, Data Models & Technology Stack

Generated: 2026-04-28 02:40 UTC

Classification: CONFIDENTIAL

1. System Overview

The FX Trading Tracker is a full-stack web application for managing foreign exchange deal tickets. It supports a three-role workflow: Traders create deal tickets, Treasury processes them (confirm/return), and Admins manage users, reference data, and audit trails. The platform is designed for on-premise deployment with switchable storage and database backends.

2. System Architecture

2.1 High-Level Architecture

The application follows a client-server architecture with a React single-page application (SPA) frontend communicating with a FastAPI REST backend via JSON over HTTPS. All API routes are prefixed with /api and routed through a Kubernetes Ingress controller.

Layer	Technology	Version	Description
Frontend	React SPA	19.0.0	Single-page app with lazy loading and route-based code splitting
API Server	FastAPI	0.110.1	Async REST API with Pydantic validation
ASGI Server	Uvicorn	0.25.0	Production ASGI server with hot-reload in dev
Database (Dev)	MongoDB	7.0.31	Document database via motor async driver
Database (Prod)	Couchbase Enterprise	SDK 4.6.0	Distributed NoSQL with scopes/collections
Storage (Dev)	Emergent Object Storage	API v1	Cloud-based file storage
Storage (Prod)	S3-Compatible (Huawei OBS)	boto3 1.42.58	On-premise object storage
Process Manager	Supervisor	-	Manages frontend (port 3000) and backend (port 8001)

2.2 Runtime Environment

Component	Version
Node.js	20.20.2
Python	3.11.15
MongoDB	7.0.31
Couchbase Enterprise SDK	4.6.0

3. Backend Technology Stack

Package	Version	Purpose
fastapi	0.110.1	Web framework, REST API
uvicorn	0.25.0	ASGI server with hot-reload
starlette	0.37.2	ASGI toolkit (CORS, routing)
pydantic	2.12.5	Request/response validation, data models
motor	3.3.1	Async MongoDB driver
pymongo	4.5.0	MongoDB synchronous operations
couchbase	4.6.0	Couchbase Enterprise SDK (KV, N1QL, scopes/collections)
boto3	1.42.58	S3-compatible object storage (Huawei OBS, MinIO, AWS)
python-jose	3.5.0	JWT token encode/decode (HS256)

Package	Version	Purpose
passlib	1.7.4	Password hashing framework
bcrypt	4.1.3	Bcrypt password hashing backend
reportlab	4.4.10	PDF report generation with branded layout
python-dotenv	1.2.1	Environment variable loading from .env
python-multipart	0.0.22	Multipart file upload handling

4. Frontend Technology Stack

4.1 Core Dependencies

Package	Version	Purpose
react	19.0.0	UI framework
react-dom	19.0.0	DOM rendering
react-router-dom	7.5.1	Client-side routing with lazy loading
axios	1.8.4	HTTP client with JWT interceptor
tailwindcss	3.4.17	Utility-first CSS framework
lucide-react	0.507.0	Icon library (200+ icons)
sonner	2.0.3	Toast notifications
recharts	3.6.0	Dashboard charts and data visualization
date-fns	4.1.0	Date formatting and manipulation
zod	3.24.4	Schema validation
react-hook-form	7.56.2	Form state management
react-day-picker	8.10.1	Date picker component
@craco/craco	7.1.0	CRA config override (path aliases)
class-variance-authority	0.7.1	Component variant styling
clsx	2.1.1	Conditional CSS classnames
tailwind-merge	3.2.0	Tailwind class conflict resolution

4.2 Shadcn/UI (Radix Primitives)

The UI component library is built on Radix UI primitives with Tailwind CSS styling (Shadcn pattern). All components are located in `/app/frontend/src/components/ui/`.

Package	Version
@radix-ui/react-dialog	1.1.11
@radix-ui/react-select	2.2.2
@radix-ui/react-dropdown-menu	2.1.12
@radix-ui/react-tabs	1.1.9
@radix-ui/react-popover	1.1.11
@radix-ui/react-label	2.1.4
@radix-ui/react-checkbox	1.2.3
@radix-ui/react-scroll-area	1.2.6
@radix-ui/react-tooltip	1.2.4
@radix-ui/react-separator	1.1.4
@radix-ui/react-switch	1.2.2
@radix-ui/react-slot	1.2.0
@radix-ui/react-accordion	1.2.8
@radix-ui/react-avatar	1.1.7

Package	Version
@radix-ui/react-toast	1.2.11
@radix-ui/react-progress	1.1.4
@radix-ui/react-radio-group	1.3.4
@radix-ui/react-alert-dialog	1.1.11
@radix-ui/react-hover-card	1.1.11
@radix-ui/react-collapsible	1.1.8
@radix-ui/react-context-menu	2.2.12
@radix-ui/react-menubar	1.1.12
@radix-ui/react-navigation-menu	1.2.10
@radix-ui/react-slider	1.3.2
@radix-ui/react-toggle	1.1.6
@radix-ui/react-toggle-group	1.1.7
@radix-ui/react-aspect-ratio	1.1.4

5. Database Design

5.1 Switchable Backend

The database layer is abstracted behind a factory pattern (`services/database.py`). The `DB_TYPE` environment variable controls which backend is active. Both backends expose identical MongoDB-like APIs (`find`, `insert`, `update`, `delete`, `aggregate`) to the application layer.

5.2 Couchbase Scope/Collection Mapping

Following Couchbase best practices, data is organized into logical scopes with separate collections. Primary and secondary indexes are auto-created on first startup.

Scope	Collection	MongoDB Equivalent	Description
identity	users	users	User accounts, credentials, roles
trading	deals	deals	FX deal tickets with full lifecycle
trading	deal_counters	counters	Sequential deal reference counters
reference	companies	companies	Client/counterparty companies
reference	banks	banks	Banking institutions with SWIFT codes
reference	bank_accounts	bank_accounts	Bank account numbers per bank
reference	currencies	currencies	Fiat, crypto, and stablecoin currencies
reference	transaction_types	transaction_types	Today, Tomorrow, Spot
reference	transfer_types	transfer_types	FX Crypto, FX Local, PDAX WD
audit	audit_logs	audit_logs	Full audit trail of all actions

5.3 Data Models

Collection: users

Field	Type	Description
id	string (UUID)	Unique identifier
email	string	Login email (unique)
first_name	string	First name
last_name	string	Last name
password_hash	string	Bcrypt hashed password
role	string	admin trader treasury
is_active	boolean	Account enabled flag
created_at	string (ISO 8601)	Creation timestamp

Collection: deals

Field	Type	Description
id	string (UUID)	Unique identifier
reference_number	string	Sequential ref (FX-YYYYMMDD-NNNN)
transaction_type	string	Today Tomorrow Spot
transfer_type	string	FX Crypto FX Local PDAX WD
deal_date	string (YYYY-MM-DD)	Deal execution date

Field	Type	Description
value_date	string (YYYY-MM-DD)	Settlement value date
client_name	string	Client/counterparty name
from_type / to_type / ours_type	string	bank crypto
from_company / to_company	string	Company name
from_bank / to_bank / ours_bank	string	Bank code
from_account_num / to_account_num / ours_account_num	string	Account number
from_wallet_address / to_wallet_address / ours_wallet_address	string	Crypto wallet
buy_currency / sell_currency	string	Currency code
currency_amount	float	Buy currency amount
amount	float	Settlement amount
rate	float	Exchange rate
status	string	pending confirmed returned cancelled
remarks	string	Trader remarks
treasury_remarks	string	Treasury processing notes
settlement_proofs	array	Array of proof objects {id, path, filename, proof_type}
history	array	Array of history entries {action, user_name, changes, timestamp}
created_by / created_by_name	string	Creator user ID and name
processed_by / processed_by_name	string	Treasury processor ID and name
created_at / updated_at	string (ISO 8601)	Timestamps

Collection: audit_logs

Field	Type	Description
id	string (UUID)	Unique identifier
action	string	Action type (deal_created, deal_confirmed, etc.)
entity_type	string	Entity type (deal, user, bank_account)
entity_id	string	Entity UUID
entity_ref	string	Human-readable reference
user_id / user_name / user_role	string	Acting user details
details	string	Human-readable description
metadata	object	Additional structured data
created_at	string (ISO 8601)	Action timestamp

6. Storage Abstraction Layer

File storage (settlement proofs) is abstracted via services/storage.py with two implementations:

Backend	Class	Protocol	Use Case
emergent	EmergentStorage	Emergent Object Storage REST API	Cloud dev environment
s3	S3Storage	S3 API via boto3 (s3v4 signature, path-style)	On-premise (Huawei OBS, MinIO, AWS)

7. Environment Variables

Variable	Purpose	Example
MONGO_URL	MongoDB connection string	mongodb://localhost:27017
DB_NAME	MongoDB database name	test_database
DB_TYPE	Database backend selector	mongodb couchbase
CB_CONNECTION_STRING	Couchbase cluster address	couchbases://cb.xxx.cloud.couchbase.com
CB_USERNAME	Couchbase auth username	fxtrading_app
CB_PASSWORD	Couchbase auth password	(secret)
CB_BUCKET_NAME	Couchbase bucket name	db_fxtrading
JWT_SECRET	JWT signing key	(secret)
CORS_ORIGINS	Allowed CORS origins	*
EMERGENT_LLM_KEY	Emergent storage init key	sk-emergent-xxx
STORAGE_TYPE	Storage backend selector	emergent s3
S3_ENDPOINT_URL	S3-compatible endpoint URL	http://localhost:9000
S3_ACCESS_KEY_ID	S3 access key ID	minioadmin
S3_SECRET_ACCESS_KEY	S3 secret access key	(secret)
S3_BUCKET_NAME	S3 bucket name	fx-trading-tracker
S3_REGION	S3 region identifier	us-east-1

8. API Endpoint Reference

All endpoints are prefixed with /api. Authentication is via Bearer JWT token in the Authorization header. Token expiry: 24 hours.

Method	Endpoint	Access	Description
POST	/api/auth/login	All	Authenticate and receive JWT token
GET	/api/auth/me	All	Get current user profile
PUT	/api/auth/change-password	All	Change own password
GET	/api/users	Admin	List users with search and pagination
POST	/api/users	Admin	Create new user account
PUT	/api/users/{user_id}	Admin	Update user details
DELETE	/api/users/{user_id}	Admin	Delete user account
GET	/api/deals	Trader/Treasury/Admin	List deals with filters and pagination
POST	/api/deals	Trader	Create new deal ticket
GET	/api/deals/{deal_id}	Trader/Treasury/Admin	Get deal detail with history and proofs
PUT	/api/deals/{deal_id}/process	Treasury	Confirm or return a pending deal
PUT	/api/deals/{deal_id}/cancel	Trader	Cancel own pending deal
PUT	/api/deals/{deal_id}/resubmit	Trader	Resubmit a returned deal
PUT	/api/deals/{deal_id}/edit	Trader	Edit a returned deal's fields
GET	/api/deals/export	Trader/Treasury/Admin	Export deals as CSV
POST	/api/deals/{deal_id}/upload	All	Upload settlement proof (image/PDF, max 10MB)
GET	/api/files/{path}	Public	Download uploaded file by storage path
DELETE	/api/deals/{deal_id}/proofs/{proof_id}	All	Delete settlement proof
GET	/api/reference/{entity_type}	All	List reference items (companies, banks, currencies, etc.)
POST	/api/reference/{entity_type}	Admin	Create reference item
PUT	/api/reference/{entity_type}/{item_id}	Admin	Update reference item
DELETE	/api/reference/{entity_type}/{item_id}	Admin	Delete reference item
GET	/api/reference/banks/{bank_id}/accounts	All	List bank accounts for a bank
POST	/api/reference/banks/{bank_id}/accounts	All	Add bank account
PUT	/api/reference/banks/{bank_id}/accounts/{account_id}	Admin	Update bank account
DELETE	/api/reference/banks/{bank_id}/accounts/{account_id}	Admin	Delete bank account
GET	/api/dashboard/stats	All	Dashboard statistics and aggregation
GET	/api/audit-logs	Admin	Audit trail with filters and pagination
GET	/api/reports/deal-blotter	All	Deal Blotter (format=json csv pdf)
GET	/api/reports/settlement	All	Settlement Report (format=json csv pdf)
GET	/api/reports/open-positions	All	Open Positions (format=json csv pdf)
GET	/api/reports/audit-trail	Admin	Audit Trail Report (format=json csv pdf)
GET	/api/reports/user-activity	Admin	User Activity Report (format=json csv pdf)
GET	/api/reports/volume-summary	All	Volume Summary (format=json csv pdf, group_by)
GET	/api/reports/client-activity	All	Client Activity Report (format=json csv pdf)
GET	/api/storage/status	Admin	Storage backend status and config
GET	/api/database/status	Admin	Database backend status and config

9. Reports System

Seven industry-standard FX trading reports are available via the /reports page. Each report supports three output formats: JSON (for table preview), CSV (download), and PDF (branded download with company colors).

Report	Description	Filters	Access
Deal Blotter	Complete log of all executed deals	Date range, status, client, currency	All roles
Settlement Report	Deals grouped by value date with bank details and proof status	Date range	All roles
Open Positions	Pending deals grouped by currency pair showing net buy/sell volume (current snapshot)	Currency pair	All roles
Transaction Audit Trail	Full history of every deal action with timestamps and change details	Date range	Admin only
User Activity	Per-user breakdown of deals created, processed, returned, pending	Date range	Admin only
Volume Summary	Deal counts and volumes grouped by day, week, or month	Date range, group by	All roles
Client Activity	Per-client deal volume, frequency, average deal size, currency pairs	Date range	All roles

9.1 PDF Branding

Element	Color	Hex
Header bar / headings	Navy	#08263e
Accent line / highlights	Red	#ec474e
Subheadings / links	Blue	#518dca
Alternating row background	Light gray	#f1f2f2

10. Security Architecture

Authentication: JWT (HS256) with 24-hour expiry. Passwords hashed with bcrypt (passlib). Role-based access control (RBAC) enforced at each endpoint. CORS configurable via CORS_ORIGINS env var. File uploads validated by content type and size (max 10MB). Settlement proof paths use UUID segments to prevent enumeration.

Security Control	Implementation
Authentication	JWT HS256, 24h expiry, stored in localStorage
Password Storage	bcrypt via passlib 1.7.4 / bcrypt 4.1.3
Authorization	Role-based (admin, trader, treasury) checked per endpoint
CORS	Configurable via CORS_ORIGINS environment variable
File Upload	Content-type whitelist, 10MB size limit, UUID path segments
API Transport	HTTPS via Kubernetes Ingress TLS termination
Token Injection	Axios interceptor auto-attaches Bearer token to all requests